

Explain Depth First Traversal with example. [L2,CO6] [10M]

Ans:

DEPTH FIRST SEARCH:

- **DFS (Depth First Search) is a graph/traversal technique where you explore as deep as possible before backtracking.**
- DFS starts from a node, visits it, then recursively visits one unvisited neighbor at a time, which produces a spanning tree as a final result.
- Spanning tree is a graph without loops.
- It use stack data structure to implement DFS Traversal.

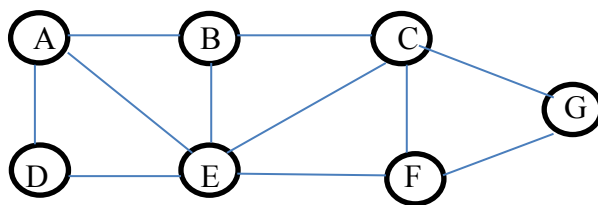
Algorithm/ steps to do DFS:

FS follows the **LIFO (Last In First Out)** principle using a **stack** (or recursion):

1. Start from a source vertex.
2. Mark the vertex as **visited** and push into stack
3. Visit any one of its **unvisited adjacent vertices of a vertex**
4. Repeat the process until **all nodes are visited**.
5. When a vertex has **no unvisited neighbors**, then use backtracking and pop one vertex from the stack.
6. Repeat steps 2,3,4 until stack becomes empty.
7. When stack becomes empty ,produce final spanning tree by removing unused edges from the graph.

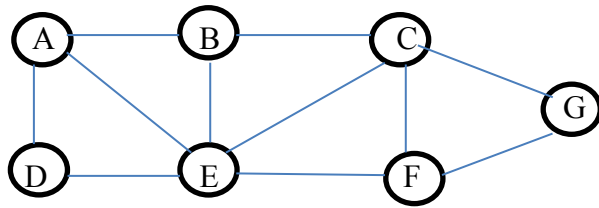
Example:

Consider the graph for DFS Traversal Produce spanning tree and path

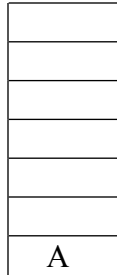


Sol:

Given graph

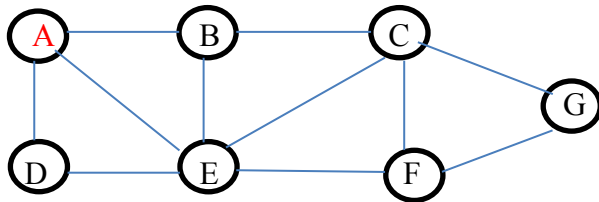


Step1: select the vertex A as starting point(Visit A).push A into the stack.



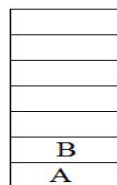
stack

output: A



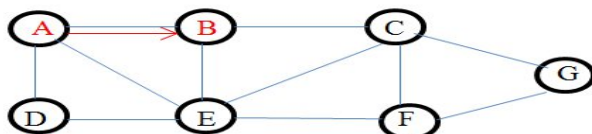
Step 2: : visit any one adjacent vertex of A which not visited(B).

Push B onto the stack.

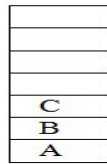


stack

output: A,B

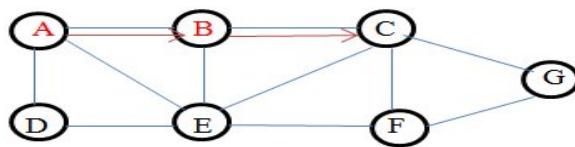


Step3: visit any one adjacent vertex of B which not visited(C).
Push C onto the stack.

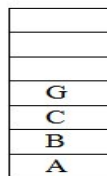


Stack

output: A,B,C

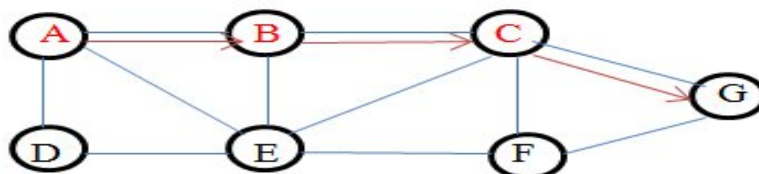


Step4: visit any one adjacent vertex of C which not visited(G).
Push G onto the stack.



stack

output: A,B,C,G

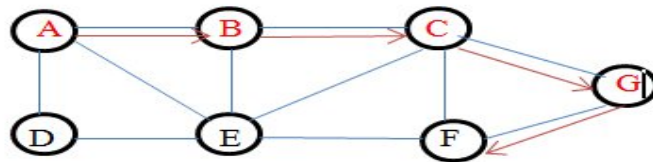


Step5: visit any one adjacent vertex of G which not visited(F).
Push F onto the stack.

F
G
C
B
A

stack

output: A,B,C,G,F

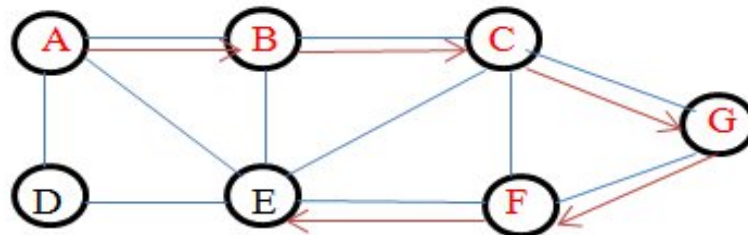


Step6: visit any one adjacent vertex of F which not visited(E).
Push E onto the stack.

E
F
G
C
B
A

stack

output: A,B,C,G,F,E

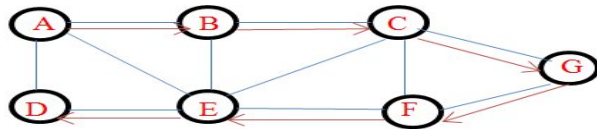


Step7: visit any one adjacent vertex of E which not visited(D).
Push D onto the stack.

D
E
F
G
C
B
A

stack

output: A,B,C,G,F,E,D

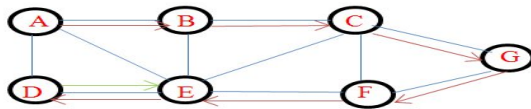


Step8: There is no new vertex to be visited from D. so backtrack.
Pop D from stack.

E
F
G
C
B
A

stack

output: A,B,C,G,F,E,D

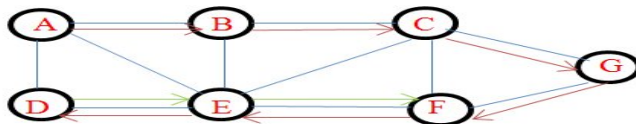


Step9: There is no new vertex to be visited from E. so backtrack.
Pop E from stack.

F
G
C
B
A

stack

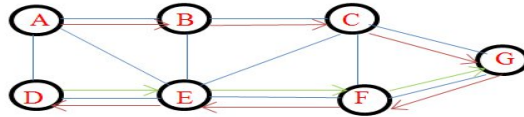
output: A,B,C,G,F,E,D



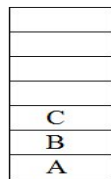
Step10: There is no new vertex to be visited from F. so backtrack.
Pop F from stack.

stack

output: A,B,C,G,F,E,D

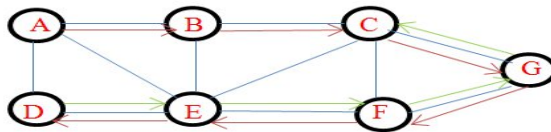


Step11: There is no new vertex to be visited from G, so backtrack.
Pop G from stack.

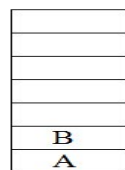


stack

output: A,B,C,G,F,E,D

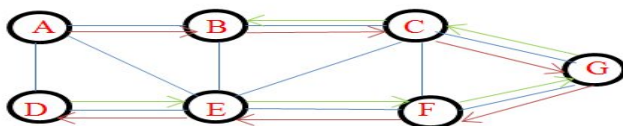


Step12: There is no new vertex to be visited from C. so backtrack.
Pop C from stack.

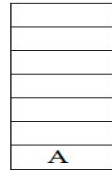


stack

output: A,B,C,G,F,E,D

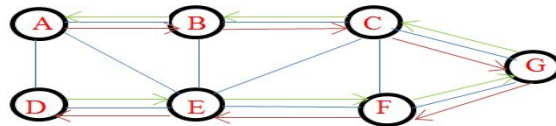


Step13: There is no new vertex to be visited from B. so backtrack.
Pop B from stack.

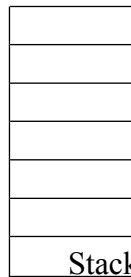


stack

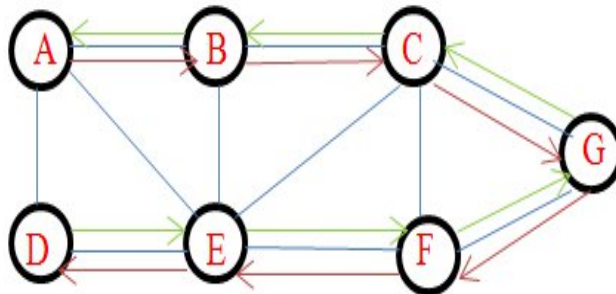
output: A,B,C,G,F,E,D



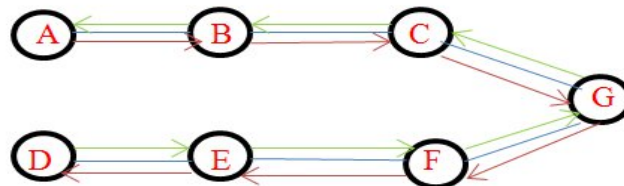
Step14: There is no new vertex to be visited from A. so backtrack.
Pop A from stack.



Stack becomes empty



Therefore stack becomes empty and final spanning tree for DFS traversal is given below.
Final DFS spanning tree:



DFS path order is:
A-B-C-G-F-E-D

